Connecting the Second Brain: Step-by-Step Integration Guides

Claude Code · Obsidian · Git · Zotero · Mendeley · Claude in Chrome · NotebookLM · Jev · Google connectors · Skills · Pipeline tools

Prepared by Isa Haskologlu, PhD · © 2026 · CC BY 4.0

2026-09-29 (pipeline-tools guide added 2026-10-06; Windows + WSL variant of the Jev guide added 2026-10-09; guides renumbered in setup order 2026-10-13)

Contents

How to Use These Guides 1

Guide 0 · Claude Code (the foundation) 3

Guide 1 · Obsidian (the knowledge base) 3

Guide 2 · Git (history and safety net) 5

Guide 3 · Source-pipeline tools (PDF, OCR and Word/PDF output) 5

Guide 4 · Jev (calibrated judgments) 8

Guide 5 · Skills and plugins 12

Guide 6 · Claude in Chrome (browser automation) 13

Guide 7 · Google Drive, Gmail and Calendar (claude.ai connectors) 13

Guide 8 · NotebookLM (via Claude in Chrome) 13

Guide 9 · Zotero, part 1: read and import 14

Guide 10 · Zotero, part 2: write-back (Research Design Profiles) 15

Guide 11 · Mendeley (planned, not yet run) 16

Final Checklist 16

License and How to Cite 17

How to Use These Guides

Each guide follows the same pattern:

- **What you get:** why the connection is worth making.

- **Status in this vault:** *working*, *working with limits*, or *planned (not yet run)*.
- **Before you start:** accounts and software you need.
- **Steps:** numbered. Each step names **where** you type (Terminal, inside Claude Code, a browser, or an app) and ends with a **check**, so you know it worked before moving on.
- **If something goes wrong:** the symptoms actually seen, and their fixes.
- **Sources:** where each instruction comes from.

Two places to type commands. This causes most setup errors, so it's worth stating once:

	Normal Terminal	Inside Claude Code
Prompt looks like	ends with `%` or `$`	starts with the Claude chevron
Commands	shell commands, no slash (`claude plugin list`, `git status`)	slash commands (`/plugin`, `/mcp`, `/chrome`) and plain-English requests
Run a shell command from here	(you are already here)	prefix it with `!`

Three security rules that apply to every guide:

1. **Never paste a key, token or password into a chat with Claude**, a screenshot, a shared document or the text of a command. Enter secrets at a hidden prompt (`read -s`) in the normal Terminal.
2. **Keep secret files out of version control.** Add the file to `.gitignore` *before* you create it, then confirm with `git check-ignore`.
3. **Prefer read-only access**, and grant write access only where a workflow needs it (Zotero write-back is the one case in this vault).

Where the facts come from. Steps marked *[vault]* come from this vault's files and recorded sessions. Steps marked *[docs]* come from official documentation checked on 2026-09-29 (URLs in each guide). Interfaces change, so if a button name differs, trust the live screen and the official docs.

Integration Map

#	Integration	Status	Needs your sign-in?	Setup time (approx.)
0	Claude Code	working	yes (Claude account)	10 min
1	Obsidian vault	working	no	10 min
2	Git version control	working	no (yes for GitHub)	10 min
3	Source-pipeline tools (PDF, OCR, Word/PDF output)	working	no	20–30 min
4	Jev (TypeSafe); Windows + WSL variant	working	yes (API key)	15 min
5	Skills and plugins	working	no	5 min
6	Claude in Chrome	working	yes (Claude login)	10 min
7	Google Drive, Gmail, Calendar	available, not authorised	yes (Google OAuth)	5 min each
8	NotebookLM	working with limits	yes (Google)	10 min

#	Integration	Status	Needs your sign-in?	Setup time (approx.)
9	Zotero: read and import	working	no	10 min
10	Zotero: write-back	working	yes (API key)	10 min
11	Mendeley	**planned, not yet run**	yes (OAuth)	30 min

Setup times are estimates, not measurements.

Guide 0 · Claude Code (the foundation)

What you get. The AI assistant that reads and writes the vault's files, runs the pipeline scripts and calls every other integration.

Status: working. **Before you start:** macOS 13 or later (or Windows 10+, Ubuntu 20.04+); a Pro, Max, Team, Enterprise or Console account. The free claude.ai plan does not include Claude Code. *[docs]*

1. **Install** (Terminal):

   ```
   curl -fsSL https://claude.ai/install.sh | bash
   ```

   Alternative: `brew install --cask claude-code` (Homebrew installs do not auto-update). *[docs]* **Check:** open a *new* Terminal window and run `claude --version`. A version number such as `2.1.x (Claude Code)` means it worked. *[docs]*

2. **Diagnose** (Terminal): `claude doctor` prints read-only install and settings diagnostics. *[docs]*

3. **Start in the vault folder and log in** (Terminal):

   ```
   cd "/path/to/Your Vault"
   claude
   ```

   Follow the browser login prompt. *[docs]* **Check:** Claude answers inside the vault. Ask it: "Read CLAUDE.md and tell me which file has the highest authority."

If something goes wrong. `claude: command not found` → the install folder isn't on your PATH yet; open a new Terminal, or see the troubleshooting page. *[docs]*

Sources. code.claude.com/docs/en/setup (checked 2026-09-29).

Guide 1 · Obsidian (the knowledge base)

What you get. A human-friendly window onto the same plain Markdown files Claude works on: browsing, search, backlinks and the graph view. Obsidian adds no AI here; it is the interface. *[vault]*

Status: working. **Before you start:** Obsidian installed from obsidian.md.

1. **Create the folder structure** (Terminal, or ask Claude to do it):

   ```
   mkdir -p "Second Brain"/{00-System,01-Library/{Originals,OCR,Text,Reports,Processed}
    ↪  ,02-Wiki,03-Projects,04-Templates,05-Inbox,06-Archive}
   touch "Second Brain"/{index.md,log.md,CLAUDE.md,RESEARCH_PROTOCOL.md}
   ```

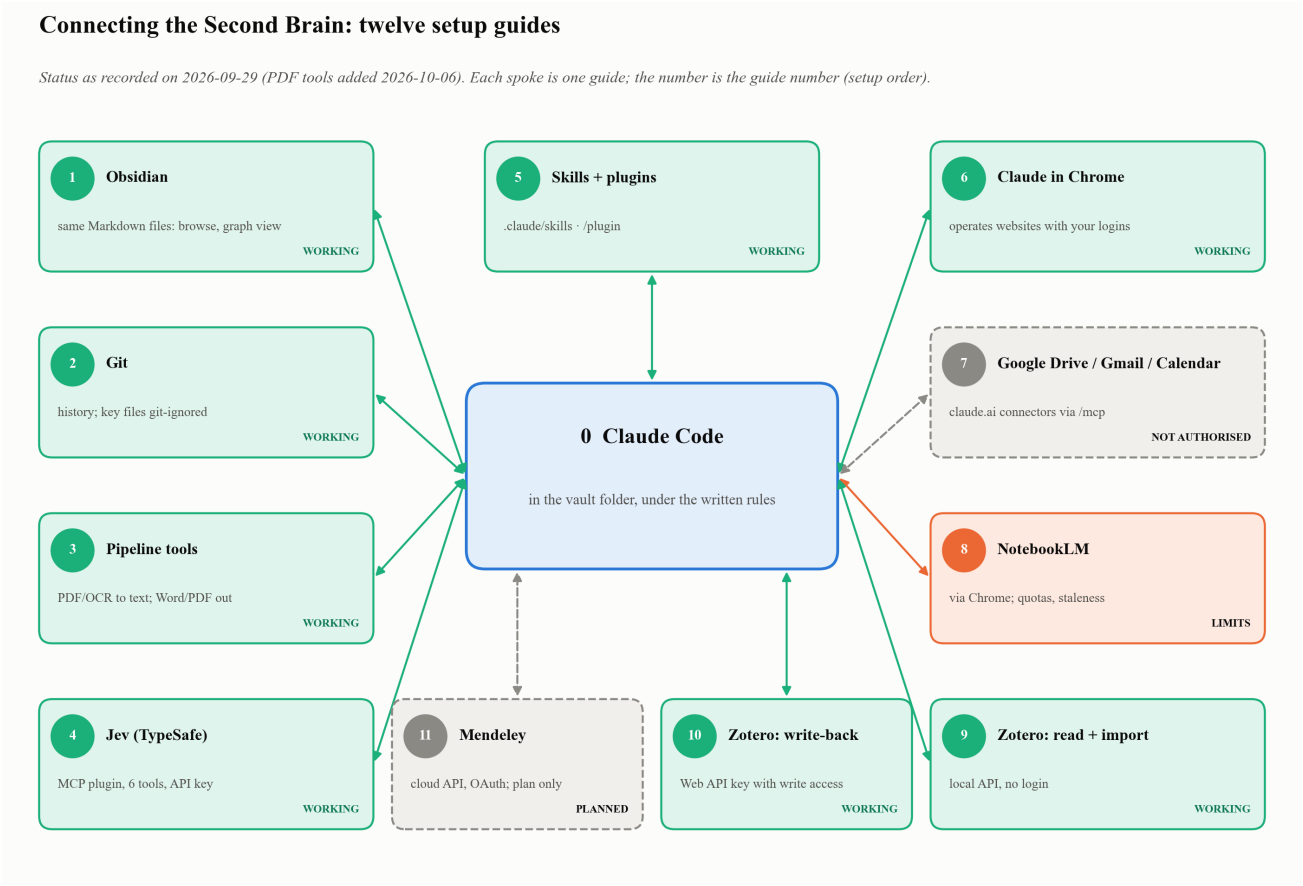


Figure 1: What connects to what

Check: `ls "Second Brain"` shows the folders. *[vault: folder structure]*

2. **Open the folder as a vault** (Obsidian app): **Vault profile** (bottom left) → **Manage Vaults…** → next to **Open folder as vault**, select **Open** → choose the folder → **Open**. *[docs]* **Check:** the folders appear in Obsidian's file explorer.
3. **Write the rulebook** (inside Claude Code): ask Claude to draft `CLAUDE.md` (workflows, page template with YAML frontmatter, naming and safety rules) and `RESEARCH_PROTOCOL.md` (no invented citations; separate source claims from interpretation). Review both yourself. *[vault]* **Check:** a new Claude Code session, asked "What is the default status of a new wiki page?", answers `needs-verification`.
4. **Add the router and contract**: `00-System/START-HERE.md` (which files to load for which task) and `00-System/ENGINEERING-CONTRACT.md` (memory types, write gate, workflow graphs, link rules). *[vault]*
5. **Use the graph view** (Obsidian): open the graph view to see `[[wikilinks]]` as a network. *[vault]*

Keep in mind. Obsidian and Claude edit the same files. Avoid editing a note in Obsidian while Claude is writing that same note.

Sources. obsidian.md/help/Files+and+folders/Manage+vaults (checked 2026-09-29); vault structure as of 2026-09-29.

Guide 2 · Git (history and safety net)

What you get. Every change to every note is recorded and reversible. This vault uses Git for its full history. *[vault]*

Status: working. **Before you start:** Git installed (`git --version`).

1. **Initialise** (Terminal, inside the vault): `git init`
2. **Protect secrets first** (Terminal): create `.gitignore` *before* any key file exists. Include at least the key files you will create (for example `00-System/Config/.zotero-web-api-key`) and generated intermediates. *[vault]* **Check:** `git check-ignore -v <path-to-key-file>` prints the matching rule.
3. **First commit** (Terminal): `git add -A && git commit -m "Initial vault"` **Check:** `git log --oneline` shows the commit.
4. **Work in branches for projects** (optional): this vault keeps a project's work on its own branch until it is ready. *[vault]*

If something goes wrong. A secret was committed → rotate the secret at its provider first, then clean the history. Deleting the file alone is not enough.

Guide 3 · Source-pipeline tools (PDF, OCR and Word/PDF output)

What you get. The free command-line programs that the vault's scripts call when a new PDF arrives: they read a PDF's text layer, run OCR (optical character recognition: turning a scanned page image into text) on scanned books, add page markers so every note can cite a page, and check the text quality. Pandoc then turns Markdown write-ups into Word and PDF files. You install them once; after that, Claude runs them for you. *[vault]*

Status: working. On 2026-10-06, `process-source.py --doctor` found all four required tools in this vault. **Before you start:** a Mac, Guide 0 (Claude Code) and Guide 1 (the vault folder). Most of the time is download time. *Times are estimates.*

Do you need this guide? Yes, if you will add PDFs or scanned books to the library. You can skip it if you only use Claude to write, or only import Zotero items that already have clean text, and come back the first time a script reports a missing tool.

Tool	What it does here	Used by	Needed?
Homebrew	Installs everything else on a Mac	(installer)	yes
poppler (`pdftotext`, `pdfinfo`)	Reads a PDF's text and page count	`inspect-pdf.py`, `process-source.py`, `validate-text.py`	yes
Tesseract + OCRmyPDF	OCR for scanned PDFs	`process-source.py`	yes, for scanned sources
qpdf	Splits books over about 100 pages into chunks	`split-pdf-qpdf.sh`	for long books
Python 3.11+ and four packages	Runs the vault's scripts	every script in `00-System/Scripts/`	yes
pandoc (+ BasicTeX for PDF)	Markdown → Word / PDF	`md-to-docx.sh`, deliverable builds	for Word/PDF output

All commands below go in the **normal Terminal** (prompt ends with `%` or `$`), not inside Claude Code. Copy one block at a time and wait for it to finish. In the PDF, a long command may wrap onto a second line marked □; it is still one command, so copy or type it as one line.

1. **Install Homebrew** (Terminal). *[docs]*

   ```
   /bin/bash -c "$(curl -fsSL
    ↪  https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
   ```

   It asks for your Mac password (nothing appears as you type; that is normal). At the end it prints **"Next steps"** with two or three commands. Run those, then open a new Terminal window.

   Check: `brew --version` prints a version number.

2. **Install the PDF tools** (Terminal). *[docs]*

   ```
   brew install poppler qpdf
   ```

   Check: `pdfinfo -v` and `qpdf --version` each print a version.

3. **Install OCR** (Terminal). OCRmyPDF's Homebrew package also installs Tesseract and the other programs it needs, with English included. *[docs]*

   ```
   brew install ocrmypdf
   ```

   For any other language (Turkish, Arabic, French …), add all Tesseract language packs. This is a large download. *[docs]*

   ```
   brew install tesseract-lang
   ```

   Check: `ocrmypdf --version` prints a version, and `tesseract --list-langs` lists `eng` plus the languages you need (`tur`, `ara`, …).

4. **Install Python** (Terminal). *[docs]*

   ```
   brew install python
   ```

Check: `python3 --version` shows 3.11 or newer. If it shows an older version, another Python comes first on your PATH (for example Anaconda). `which python3` shows which one; use `$(brew --prefix)/bin/python3` in step 5 instead of `python3`.

5. **Create the vault's own Python environment and install the packages** (Terminal, inside the vault folder). Homebrew's Python refuses `pip install` outside a virtual environment, so the vault gets its own environment in a hidden `.venv` folder. Obsidian does not show it; add it to `.gitignore` so Git ignores it too. *[docs]*

   ```
   cd "/path/to/Your Vault"
   echo ".venv/" >> .gitignore
   python3 -m venv .venv
   source .venv/bin/activate
   python -m pip install -r 00-System/requirements.txt
   ```

   `requirements.txt` lists PyYAML, requests, pypdf and pytesseract. *[vault]*

   Check: `python -c "import yaml, requests, pypdf, pytesseract; print('ok')"` prints `ok`. A "[notice] A new release of pip is available" message is harmless; you can ignore it.

6. **Tell Claude which Python to use** (inside Claude Code). Claude starts its own shell, which does not activate `.venv`. Say: *"Add a rule to CLAUDE.md: run the vault's Python scripts with `.venv/bin/python`."*

   Check: ask *"Which Python do you use for the vault's scripts?"* The answer names `.venv/bin/python`.

7. **Run the pipeline health check** (Terminal, in the vault).

   ```
   .venv/bin/python 00-System/Scripts/process-source.py --doctor
   ```

   Check: it lists `pdftotext`, `pdfinfo`, `ocrmypdf` and `tesseract` under "Available tools" and ends with **"All dependencies available"**. *[vault]*

8. **Install pandoc for Word output** (Terminal). *[docs]*

   ```
   brew install pandoc
   ```

   Check: `pandoc --version` prints a version.

9. **Optional: PDF output.** Pandoc needs a LaTeX engine to make PDFs. Pandoc's documentation recommends the small BasicTeX over the 4 GB MacTeX. *[docs]*

   ```
   brew install --cask basictex
   ```

   Then open a new Terminal window.

   Check: `xelatex --version` prints a version. If a PDF build fails with font errors, pandoc's docs suggest `tlmgr install collection-fontsrecommended` (prefix it with `sudo` if it reports a permission error).

10. **First real test** (Terminal). Put any PDF in `05-Inbox/` and run the read-only inspector. `--dry-run` prints the report without saving anything:

    ```
    .venv/bin/python 00-System/Scripts/inspect-pdf.py --file "05-Inbox/your-file.pdf"
    ↪  --dry-run
    ```

    Check: an "Intake Report" appears with the page count and a text-layer assessment ("usable text" or "OCR required"). From now on you can simply ask Claude to `inspect raw` or to process a source. *[vault]*

If something goes wrong.

- `brew: command not found` right after installing → you skipped the "Next steps" commands, or the Terminal window is old. Run them, then open a new window.
- `error: externally-managed-environment` → pip ran outside the environment. Run `source .venv/bin/activate` (or use `.venv/bin/python -m pip …`) and repeat.
- The doctor lists `tesseract` or `ocrmypdf` under "Missing tools" → run `brew install ocrmypdf` again, then `brew doctor`.
- OCR output is gibberish for a non-English book → the language pack is missing or not selected. Check `tesseract --list-langs`, then pass the language, for example `--lang tur` or `--lang tur+ara`.
- `ModuleNotFoundError: No module named 'yaml'` → the script ran with the wrong Python. Use `.venv/bin/python`, not `python3`.
- `split-pdf.py` needs the older PyPDF2 package, which is not in `requirements.txt`. Use `split-pdf-qpdf.sh` instead.

Linux and Windows. On Ubuntu/Debian, the vault's scripts suggest `apt install poppler-utils tesseract-ocr ocrmypdf qpdf pandoc` plus language packs such as `tesseract-ocr-tur`. Windows was not tested with this vault.

Sources. Homebrew: brew.sh; docs.brew.sh/Language-Runtimes-and-Packages (PEP 668 and virtual environments). OCRmyPDF: ocrmypdf.readthedocs.io/en/latest/installation.html. Pandoc: pandoc.org/installing.html. Homebrew formulae: formulae.brew.sh (poppler, qpdf, basictex). Python: docs.python.org/3/library/venv.html. Checked 2026-10-06. Vault behaviour: `00-System/Scripts/process-source.py`, `inspect-pdf.py`, `zotero_doctor.py`.

Guide 4 · Jev (calibrated judgments)

Source of truth: "Installing Jev for Claude Code", audited presentation v2 (September 2026). Details marked (companion) come from its companion guide.

What you get. Bounded, repeated judgments (classify, score, check, rank, triage) with probabilities, a confidence value and an **act / review / abstain** recommendation. **Jev is a decision aid: it does not make a source true, verify a citation, or replace domain judgment.**

Jev in one minute. Jev is TypeSafe's "System One" model: where Claude reasons and writes (System 2), Jev makes one fast, typed judgment (a choice, a yes/no probability or a score) and says how sure it is. In this vault's tests a call took 141–255 ms. It fits inside the *decision* points of a workflow graph: the graph defines the roads, Jev picks the branch, and verification stays a separate step.

Status: working. **Before you start:** Claude Code; Node.js 20.12 or newer (`node --version`; the plugin runs through `npx`) (companion); a TypeSafe account.

What you install: Claude Code (where you type) → **Jev MCP plugin** (six decision tools plus an agent skill; community-maintained, not an official TypeSafe or Anthropic product) → **TypeSafe API key** (without it: `no_api_key` or `401`). The separate TypeSafe workflow skill is optional.

1. **Create an API key** (browser): `https://console.typesafe.ai/settings/keys` → create a key with a clear name → copy it once into a password manager. Use a direct TypeSafe key (companion).
2. **Add the marketplace** (inside Claude Code): `/plugin marketplace add rashedInt32/jev-mcp`. If an "Add Marketplace" box opens, enter only `rashedInt32/jev-mcp`. **Check:** "Successfully added marketplace: jev-mcp".
3. **Install** (inside Claude Code): `/plugin install jev@jev-mcp` → **Install for you (user scope)** → `/reload-plugins`. **Check:** Jev appears once, at user scope.

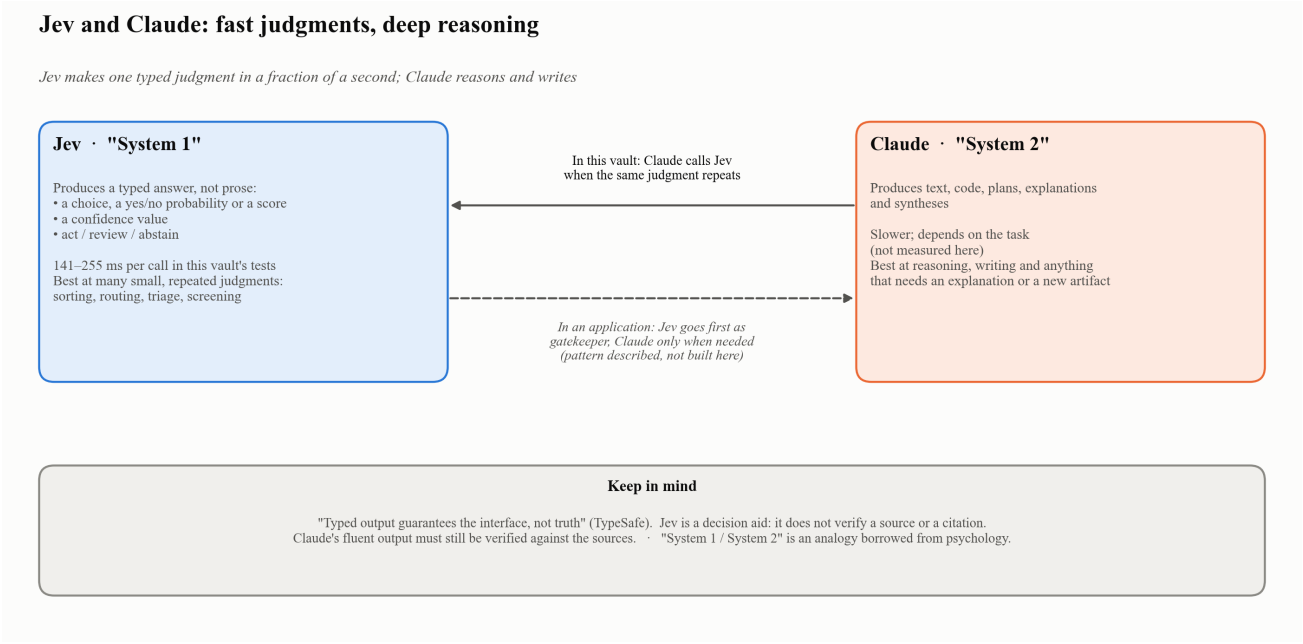


Figure 2: Jev as “System 1”, Claude as “System 2”

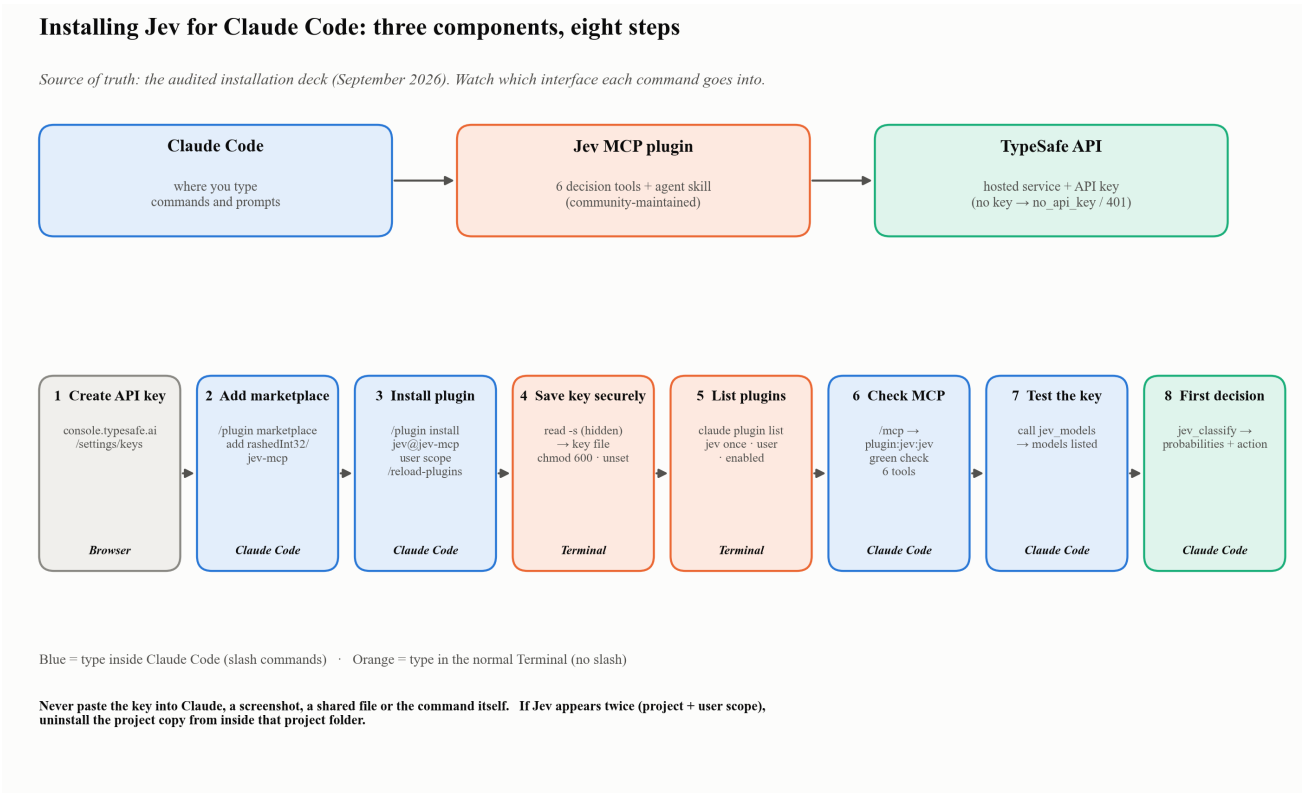


Figure 3: Installing Jev: three components, eight steps

4. **Save the key** (Terminal, one line at a time):

   ```
   mkdir -p ~/.config/typesafe
   read -s "KEY?Paste your TypeSafe API key, then press Enter: "
   printf '%s' "$KEY" > ~/.config/typesafe/key
   chmod 600 ~/.config/typesafe/key
   unset KEY
   ```

   Never type a placeholder like `YOUR_KEY`. The plugin looks for `TYPESAFE_API_KEY`, then `JEV_API_KEY`, then this file (companion).

5. **List plugins** (Terminal): `claude plugin list`. **Check:** `jev@jev-mcp` once · scope user · enabled.

6. **Check MCP** (inside Claude Code): `/mcp`. **Check:** `plugin:jev:jev`, green check, six tools.

7. **Test the key** (inside Claude Code): "Call the Jev jev_models tool and show me the available models." **Check:** `jev-latest` (recommended) and `jev-preview` are listed.

8. **First decision** (inside Claude Code): "Use Jev to classify: 'My card was charged twice and I want a refund.' Choices: billing, technical, sales, other. Show all probabilities and confidence." **Check:** billing 1.00, action **act**.

Duplicate installation. If Jev appears twice (project + user scope), remove the project copy **from inside that project folder**: `cd "/path/to/project"` → `claude plugin uninstall jev@jev-mcp --scope project` → `claude plugin list`.

Symptom	Fix
`no_api_key`	Create `~/.config/typesafe/key`, `chmod 600`, restart Claude Code
`401`	Generate a new key and replace the file contents
Red × in `/mcp`	Restart Claude Code; then `claude --debug` (check Node ≥ 20.12)
"Marketplace source invalid"	Enter only `rashedInt32/jev-mcp` in the source box
Update installed	`/reload-plugins`; restart only if the old version stays loaded

Optional rule for `CLAUDE.md`. Use Jev automatically when the same bounded judgment applies to many items, the allowed outputs are explicit, and confidence changes the next action. No arithmetic or date comparison. Never present Jev output as research evidence or verification. **Do not send** student-identifiable data, unpublished confidential research, or credentials.

Guide 4, Windows variant: VS Code + WSL (Ubuntu)

Source: the owner's own setup session on a Windows computer, 9 October 2026, guided by a ChatGPT conversation and checked against screenshots of the result [vault: user session]. Steps 1–4 were confirmed working on that computer. Step 6 was tried and did not work. Steps 7–8 were done, but whether the shortcut opens WSL every time was not yet confirmed. The ChatGPT text is not evidence; only what the screenshots showed is reported as a result.

Why a separate variant. On Windows, Claude Code can run either natively or inside WSL (the Linux subsystem). These are **two separate environments**. Each has its own Claude Code installation, plugins, API key and chat history. Only the vault files are shared, because WSL reads the same folder under `/mnt/c/…`.

	Windows (native)	WSL (Ubuntu)
Vault files (`CLAUDE.md`, `PROJECT-STATE.md`, wiki)	same folder	same folder
Claude Code, plugins, Jev	own installation	own installation
TypeSafe API key	own copy	own copy, **not inherited**
Claude chat history	stays here	starts empty

The project memory is kept, because it lives in the files. Earlier Windows conversations do **not** appear in WSL.

Which Jev package. On that computer the installed package was `jev@jev-engineering` (version 0.3.0). It adds a `/jev:stats` command, a `permission_gate` hook and a **shadow mode**. That is a different package from `jev@jev-mcp`, which the steps above install and which this vault tested. Use whichever package you installed, and check it with `claude plugin list`.

1. **Open the vault in a WSL window** (Ubuntu terminal):

   ```
   cd "/mnt/c/path/to/Your Vault"
   code .
   ```

   Check: the bottom-left corner of VS Code shows `WSL: Ubuntu-22.04`, and the terminal prompt looks like `user@host:~$`, not `PS C:\…`.

2. **Install Jev in WSL** (in this window). Repeat steps 2–7 above. The Windows installation does not carry over.

3. **Give Claude Code in WSL the TypeSafe key** (VS Code terminal, WSL). The symptom is `TypeSafe API KEY: MISSING`, even though the key works on Windows. The script below saves the key as an environment variable in `~/.claude/settings.json` and keeps your other settings. The key is typed at a hidden prompt.

   ```
   python3 - <<'PY'
   import json, getpass
   from pathlib import Path
   p = Path.home() / ".claude" / "settings.json"
   p.parent.mkdir(parents=True, exist_ok=True)
   settings = json.loads(p.read_text()) if p.exists() else {}
   settings.setdefault("env", {})["TYPESAFE_API_KEY"] = getpass.getpass("TypeSafe API
    ↪  Key: ")
   p.write_text(json.dumps(settings, indent=2) + "\n")
   p.chmod(0o600)
   print("TypeSafe API Key saved for Claude Code.")
   PY
   ```

   Security: the key is stored as plain text. `chmod 600` limits who can read it but does not encrypt it. The file is in your home folder, outside the vault, so it does not enter Git. Don't use this method if your organisation forbids keys stored in plain text. **Bash, not zsh:** Ubuntu's shell is bash, where the zsh form `read -s "KEY?…"` in step 4 above fails. If you prefer the key file, use `read -rsp "Paste your TypeSafe API key: " KEY; echo` instead.

4. **Reload and verify** (VS Code, then inside Claude Code):

   - Press Ctrl+Shift+P, choose **Developer: Reload Window**, then open a new Claude Code session.
   - Run `/jev:stats`, ask *"Run `pwd` and report the working directory. Do not read or modify project files."*, then run `/jev:stats` again.

- **Check:** the Jev call count rises and the error count does not. Observed on 9 October 2026: 1 call, decision `allow`, 427 ms, shadow mode.
- With `jev@jev-mcp`, use `/mcp` and step 7 above instead.

5. **Stay in shadow mode at first** (inside Claude Code). In shadow mode Jev records its decisions but does not change Claude Code's permission prompts. Commands and their outputs may be sent to TypeSafe. Do not let Jev control permissions for work involving confidential, student-identifiable or institutional data until you have decided what may be sent.
6. **Tried, not enough: "Window: Restore Windows = all"** (VS Code settings). VS Code reopened its windows, but the vault came back in Windows with a PowerShell prompt, not in WSL.
7. **Open WSL with one click: a desktop shortcut** (Windows).
   - Right-click the desktop → **New** → **Shortcut**.
   - As the location, enter:

     ```
     C:\Windows\System32\wsl.exe -d Ubuntu-22.04 -- bash -lc 'cd "/mnt/c/path/to/Your
     ↪  Vault" && code .'
     ```

   - Name it, for example *Second Brain (WSL)*.
   - **Check:** opening it shows `WSL: Ubuntu-22.04` at the bottom left. Use this shortcut instead of the normal VS Code icon.
8. **Pin the shortcut** (Windows). Right-click it → **Pin to Start**, which worked. For the taskbar, try right-click → **Show more options** → **Pin to taskbar**. Whether this works for a `wsl.exe` shortcut was not yet confirmed.

Symptom	Fix
`TypeSafe API KEY: MISSING` in WSL	Step 3, then reload the window
Bottom-left shows no `WSL:`; terminal shows `PS C:\`	You are in Windows. Use the shortcut from step 7
Earlier Claude conversations are missing	Expected: chat history is per environment. Project memory is in the vault files
`read: KEY?…: not a valid identifier`	That is zsh syntax in bash. Use `read -rsp "…" KEY`

Guide 5 · Skills and plugins

What you get. Packaged, repeatable procedures Claude loads when a task needs them. *[vault]*

Status: working. The vault has five working skills in `.claude/skills/`: `citation-verification`, `presentation-builder`, `process-sources`, `research-ideation` and `zotero-sources`, plus a `zotero-import` folder that is currently empty. *[vault]*

1. **Vault skills** (file system): each skill is a folder in `.claude/skills/<name>/` containing a `SKILL.md` whose frontmatter has `name` and `description`. Claude discovers them automatically in that project. *[vault]* **Check** (inside Claude Code): type `/` and look for the skill name, or ask "Which skills are available?"
2. **Marketplace plugins** (inside Claude Code): `/plugin marketplace add <owner/repo>` → `/plugin install <name>@<marketplace>` → `/reload-plugins`. This is how Jev (Guide 4) and the optional TypeSafe skill were installed. *[vault]* **Check** (Terminal): `claude plugin list`.
3. **Keep one copy per plugin.** Prefer user scope for personal tools. A project-scope copy can override the user copy inside that repository (see Guide 4).

Guide 6 · Claude in Chrome (browser automation)

What you get. Claude can operate websites in your own Chrome, with your existing logins. This is the route to tools that have no API, such as NotebookLM. *[docs; vault]*

Status: working. **Before you start:** Google Chrome (or Edge or another Chromium browser); a direct Anthropic plan (Pro, Max, Team or Enterprise); Claude Code signed in with `/login`. API-key sign-in keeps Chrome integration off. *[docs]*

1. **Install the extension** (browser): "Claude in Chrome" from the Chrome Web Store (version 1.0.36 or later). *[docs]*
2. **Launch with Chrome enabled** (Terminal): `claude --chrome`. The first time, a one-time dialog explains site permissions; press Enter. *[docs]*
3. **Check the connection** (inside Claude Code): `/chrome`. **Check:** "Status: Enabled" and "Extension: Installed". *[docs]*
4. **Optional:** in `/chrome`, choose **Enabled by default** to skip the flag. This keeps browser tools loaded and uses more context. *[docs]*
5. **Try it** (inside Claude Code): "Open notebooklm.google.com in a new tab and tell me what you see." Approve the site permission when asked. *[docs]*

If something goes wrong. *[docs]*

Symptom	Fix
Extension "Not detected"	Enable it in `chrome://extensions`; restart Chrome once after the first setup
"Receiving end does not exist"	`/chrome` → **Reconnect extension**
Commands stop responding	A JavaScript dialog may be open on the page; dismiss it, then ask Claude to continue

Sources. code.claude.com/docs/en/chrome (checked 2026-09-29).

Guide 7 · Google Drive, Gmail and Calendar (claude.ai connectors)

What you get. Claude can read documents, mail and calendar events you authorise, without browser automation.

Status: available but **not authorised** in this vault. The connectors appear in `/mcp`, but they have not been signed in. *[vault]*

1. **Open the connector list** (inside Claude Code): `/mcp`. Unused claude.ai connectors are grouped under **Show unused connectors**; select it to list them. *[vault: /mcp screenshot, 2026-09-29]*
2. **Authorise one** (inside Claude Code): select the connector (e.g., "claude.ai Google Drive") and follow the browser sign-in. **Only you can do this step**; Claude cannot start the OAuth prompt on your behalf. *[vault]*
3. **Check** (inside Claude Code): "List the files in my Drive folder *X*." Claude should return file names.
4. **Scope it.** Authorise only the connectors a project needs. Everything Claude reads through a connector enters the conversation.

Guide 8 · NotebookLM (via Claude in Chrome)

What you get. Formats Claude does not produce itself: mind maps, NotebookLM infographics and audio overviews, plus questions across a whole notebook of sources. The benefit is *formats*, not stronger sourcing:

the vault's own evidence rules are stricter. *[vault]*

Status: working with limits. There is **no API**. Claude operates the NotebookLM website through Claude in Chrome (Guide 6). First used on 2026-09-23. *[vault]*

1. **Sign in** (browser): open notebooklm.google.com in Chrome and sign in with your Google account.
2. **Open or create a notebook** (inside Claude Code): "Open NotebookLM and open my notebook *X*" (or "create a notebook named *X*").
3. **Add a source** (inside Claude Code): "Add the text of `<vault file>` as a source." In the vault's runs, **pasting the text as a source was more reliable than uploading the file**. *[vault]* **Check:** the new source appears in the Sources list.
4. **Ask or generate** (inside Claude Code): "Ask the notebook: …" or "Generate a mind map / infographic / audio overview", then "save a screenshot to `<folder>`". **Check:** the output file exists in the vault folder.
5. **Bring results back labelled.** Store NotebookLM outputs as derived material next to the project, never as evidence in `02-Wiki/`. *[vault]*

Limits seen in practice. *[vault]*

- **Usage quota:** a batch of weekly infographics stopped after the first week ("You're almost at your AI usage limit"). Plan batches, or spread them over days.
- **Staleness:** when a fact-check corrected numbers, the NotebookLM infographics made earlier still showed the old ones, and had to be deleted and regenerated. Generate NotebookLM visuals **after** verification.
- **Long waits:** generation takes minutes; Claude waits and re-checks the page.

Guide 9 · Zotero, part 1: read and import

What you get. Search your Zotero library and import PDFs *with their bibliographic metadata*, so every wiki note can cite a real record. Imports then go through the source pipeline (quality gate → notes). *[vault]*

Status: working. It runs through the **local API** of the Zotero desktop app (`localhost:23119`), which is read-only and needs no login. *[vault; docs]*

Before you start: Zotero desktop installed and open; the vault's `zotero-sources` skill (`.claude/skills/zotero-sources/`).

1. **Enable the local API** (Zotero app): **Zotero → Settings/Preferences → Advanced → General** → check **"Allow other applications on this computer to communicate with Zotero."** Restart Zotero if needed. *[vault: 00-System/Zotero/README.md]*
2. **Health check** (inside Claude Code): `/zotero-sources doctor` **Check:** it reports that the API is reachable, the vault structure is valid, and the scripts and Python dependencies are present. *[vault]*
3. **Browse** (inside Claude Code): `/zotero-sources collections`, then `/zotero-sources search "your topic"`. *[vault]*
4. **Import one item** (inside Claude Code): `/zotero-sources import --item <ITEM_KEY>`. The PDF is copied to `05-Inbox/`, and the file hash is checked after copying. Duplicates are detected by file hash, Zotero key, DOI, ISBN and title/creator/year. *[vault]*
5. **Process it** (Terminal or inside Claude Code): run the source pipeline (`process-source.py`), read the quality report, and ingest only if `quality_status` is `passed` or `usable-with-review`. *[vault]* **Check:** a `…__zotero.md` provenance record appears in `01-Library/Reports/`, next to the quality report.

If something goes wrong.

Symptom	Fix *[vault]*

Symptom	Fix *[vault]*
"Local API is not enabled"	Tick the setting in step 1 and restart Zotero
"Cannot connect to Zotero local API"	Zotero desktop must be open
"No PDF attachments found"	The item has metadata only; attach the PDF in Zotero or import it manually
"Duplicate detected"	The source is already in the vault; nothing to do

Scale note. The vault's first full run (2026-09-02) turned 51 PDF-bearing sources into about 290 wiki notes. Never import a whole library at once; the configuration requires explicit selection. *[vault]*

Guide 10 · Zotero, part 2: write-back (Research Design Profiles)

What you get. After a source is ingested, a condensed **Research Design Profile** (question, argument, theory, variables, method, data, findings, abstract, conclusion) is written back to the Zotero item as a child note. The summary then follows the reference wherever you use it. *[vault]*

Status: working (first run 2026-09-20: 7 notes, each returned HTTP 200 and one was re-fetched to confirm). *[vault]*

Why a second connection? The local API cannot write. Writing needs the **Zotero Web API** and a key with write access. *[vault; docs]*

1. **Create a key** (browser): zotero.org → Settings → **API Keys** → create a new key with **library access, notes access and write access**. *[docs: zotero.org/settings/keys/new]*
2. **Git-ignore the key file first** (Terminal):

   ```
   echo "00-System/Config/.zotero-web-api-key" >> .gitignore
   git check-ignore -v 00-System/Config/.zotero-web-api-key
   ```

3. **Save the key through a hidden prompt** (Terminal):

   ```
   read -s "ZKEY?Paste your Zotero API key: "
   printf '%s' "$ZKEY" > 00-System/Config/.zotero-web-api-key
   chmod 600 00-System/Config/.zotero-web-api-key
   unset ZKEY
   ```

4. **Verify the key's permissions** (Terminal):

   ```
   KEY=$(cat 00-System/Config/.zotero-web-api-key)
   curl -s "https://api.zotero.org/keys/$KEY" -H "Zotero-API-Key: $KEY" | python3 -m
    ↪  json.tool
   ```

   Check: the response lists your **userID** and shows library, notes and write access. *[vault; docs]*

5. **Push profiles** (inside Claude Code): ask Claude to write the profile for an ingested source to its Zotero item. Behind the scenes this is a `POST https://api.zotero.org/users/<userID>/items` with `itemType: note`, the parent item key, the `Zotero-API-Key` header and a fresh `Zotero-Write-Token`. *[vault]* **Check:** the note appears under the item in Zotero after sync; Claude re-fetches one note to confirm.

Security. Never share the key in chat or as a screenshot. It grants write access to your whole library. Revoke it at zotero.org if it is ever exposed.

Guide 11 · Mendeley (planned, not yet run)

What you get. A second reference manager feeding the same intake pipeline as Zotero.

Status: planned, not yet run. A setup plan exists (`00-System/mendeley-integration-setup-guide_20260920.pptx`), but the OAuth registration has not been completed in this vault. The steps below are that plan, corrected for the security rules above. *[vault]*

Why it is harder than Zotero. Mendeley has no local API; Mendeley Desktop has been discontinued. The only route is the cloud API (`api.mendeley.com`), which needs **OAuth 2.0** through your Elsevier account. Only you can complete that login. *[vault]*

1. **Register an app** (browser): sign in at the Mendeley developer portal (dev.mendeley.com, per the plan) → create a new app → name (e.g., "Second Brain Vault Import"), short description, and a **redirect URL** such as `http://localhost:8888/callback`. Field names on the live page may differ. *[vault: plan]*
2. **Store the Client ID and Client Secret** (Terminal): git-ignore a config file *first*, then enter the secret at a hidden prompt (as in Guide 10, step 3). **Do not paste the secret into chat.** The original plan asked for it in chat; that step is replaced here. *[vault, corrected]*
3. **One-time browser login** (browser): open the authorisation URL, log in, approve. The browser is sent to the redirect URL with `code=…` in the address bar. The "can't connect" page is expected, because nothing listens on that port. Copy the code. *[vault: plan]*
4. **Exchange the code for tokens** (inside Claude Code): Claude exchanges code + client credentials for an access token (expires after about an hour) and a refresh token (renews access automatically). Both go into the same git-ignored file. *[vault: plan]*
5. **Build and run the checks**: a `mendeley-sources` skill mirroring `/zotero-sources` (doctor / search / import), handing off to the same intake pipeline. **Check:** `doctor` confirms valid tokens *without printing them*. *[vault: plan]*

Revoke access at any time from your Mendeley/Elsevier account's app settings, then redo step 3.

Final Checklist

- ☐ `claude --version` prints a version; `claude doctor` is clean
- ☐ The vault opens in Obsidian; `CLAUDE.md` and `RESEARCH_PROTOCOL.md` exist
- ☐ `git log` shows history; `git check-ignore` confirms every key file is ignored
- ☐ `process-source.py --doctor` ends with "All dependencies available"; `pandoc --version` works
- ☐ `/mcp` shows `plugin:jev:jev` with six tools; `jev_models` succeeds
- ☐ `claude plugin list` shows each plugin once
- ☐ `/chrome` shows "Status: Enabled" and "Extension: Installed"
- ☐ Only the Google connectors you need are authorised
- ☐ NotebookLM opens through Claude in Chrome; outputs saved as derived material
- ☐ `/zotero-sources doctor` passes (Zotero open, local API enabled)
- ☐ Zotero write key verified via `api.zotero.org/keys/…` (only if you use write-back)
- ☐ Mendeley: decided whether to run the OAuth plan (optional)

License and How to Cite

© 2026 Isa Haskologlu, PhD. This work is licensed under the **Creative Commons Attribution 4.0 International License (CC BY 4.0)**: https://creativecommons.org/licenses/by/4.0/. You may share and adapt it for any purpose, provided you give appropriate credit, link to the license, and indicate if changes were made.

Suggested citation (APA 7): Haskologlu, I. (2026). *The Second Brain: An AI research system you can audit* [Guide and interactive hub]. https://claude.ai/artifact/EMh3hwYGRD6KdTSQFMEPVh